

SpectraSensML v1.0 — User Guide

SpectraSensML – USER GUIDE

OVERVIEW

=====

This software trains and compares 26+ regression models that predict a **continuous temperature-like variable** from luminescence spectra. It is designed for **luminescence thermometry**, but the variable name and unit are configurable, so it can equally be used for any scalar quantity that depends on a controllable parameter (pressure, pH, magnetic field, ...).

Pipeline at a glance:

1. **Load** spectra organised in temperature folders.
2. **Pre-process** them (crop, optional background subtraction, choice of normalisation, LIR band selection).
3. (Optional) **PCA** dimensionality reduction.
4. Run the **Benchmark** across the selected model groups.
5. Inspect the **Leaderboard** and 24 publication-ready figures with matching CSV data per sub-panel.

FILE FORMAT – WHAT EACH SPECTRUM FILE SHOULD LOOK LIKE

=====

Each spectrum is a plain-text file with **two columns** of numbers: **wavelength (nm)** and **intensity** (raw counts). The two columns must be whitespace- or comma-separated.

A typical header looks like this:

```
Spectrometer model: ...
Date: 2024-...
Integration Time (sec): 0.500
Boxcar width: 0
Number of pixels: 2048
>>>>Begin Spectral Data<<<<
899.7 1234
899.9 1241
...
```

Two markers matter:

- * **Integration-time prefix** (default ``Integration Time (sec):``) – the loader divides every intensity by this number, so spectra recorded with different exposures are still comparable. If the prefix line is missing, integration time falls back to 1.0 (no scaling).
- * **Header end marker** (default ``>>>>Begin Spectral Data<<<<``) – every line up to and including this marker is skipped. If the marker is left blank the loader auto-detects the first parseable numeric pair instead.

The same wavelength axis must be used by every spectrum (a small tolerance is allowed). If files have *only* numeric data with no header at all, clear the marker box to enable auto-detect.

FOLDER LAYOUT – HOW TO PREPARE YOUR DATA

=====

Place all your spectra under one **root folder**. Inside the root, create

SpectraSensML v1.0 — User Guide

****one subfolder per temperature****, with names following either convention:

- * ``pNNN`` – positive Celsius offset; NNN is degrees Celsius (so ``p27`` is 27 °C $\Rightarrow$ 300.15 K after adding 273.15).
- * ``mNNN`` – negative Celsius offset (``m100`` $\Rightarrow$ -100 °C $\Rightarrow$ 173.15 K).
- * Plain integer or decimal – temperature ****already in Kelvin**** (``300`` $\Rightarrow$ 300 K, ``473`` $\Rightarrow$ 473 K, ``298.15`` $\Rightarrow$ 298.15 K).

You can mix both conventions in the same root folder. Folders are sorted and displayed by their parsed temperature value.

Example layouts:

```
my_dataset/          ← Celsius-offset convention
  m050/   spec_0001.txt spec_0002.txt ...
  p020/   ...
  p080/
  p120/
  ...

my_dataset/          ← plain-Kelvin convention
  223/    spec_0001.txt ...
  300/    ...
  350/
  400/
  ...
```

The Data tab discovers all temperature folders, parses T from the name, and shows each in the ****Folder table**** where you can:

- * override the numeric T value (any unit, not just K),
- * set the ****Role**** of every folder:
 - * ****Train/Val**** – spectra of this folder participate in training and validation (split by the ``train_frac`` slider).
 - * ****test_unseen**** – every spectrum is held out for final evaluation; the model never sees these spectra during training.
 - * ****Do not use**** – folder ignored entirely.

By default any folder whose rounded integer T ends in 0 becomes Train/Val, the rest become test_unseen – a convenient automation if you acquired data on a regular 10 °C grid. You can change roles freely; click ****Apply folder roles $\rightarrow$ dataset**** after editing.

SETS – TRAIN, VAL, TEST_UNSEEN

=====

The benchmark uses three independent ****sets****:

- * ****train**** – spectra the models fit on. Drawn from Train/Val folders with probability ``train_frac`` (default 80 %).
- * ****val**** – held-out fraction of Train/Val folders. Used for hyper-parameter selection inside each model and as an honest in-domain error estimate.
- * ****test_unseen**** – every spectrum from folders marked test_unseen. These temperatures are typically ****not present in train at all**** (e.g. odd integers if you trained on multiples of 10). They probe extrapolation and interpolation between training points and are the strongest generalisation test.

SpectraSensML v1.0 — User Guide

Metric tables and figures are reported separately for all three sets.

WORKFLOW — RECOMMENDED CLICK ORDER

=====

1. ****Tab 1 · Data****
 - * Browse to your root folder.
 - * Adjust **RNG seed**, **Train fraction**, **Variable name/unit** if needed.
 - * Click ****Load Spectra****.
 - * Verify the folder table; edit T values or roles if required, then click ****Apply folder roles → dataset****.
2. ****Tab 2 · Pre-process****
 - * Pick λ min / λ max to crop the noisy edges.
 - * Choose a normalisation (SNV is a safe default).
 - * Set the I1 and I2 wavelength bands for the LIR diagnostic.
 - * Click ****Run pre-processing**** to preview the normalised mean spectra.
3. ****Tab 3 · PCA****
 - * Choose how many components to fit (e.g. 9). The screen shows explained variance and loadings.
 - * Pick the number of components to feed the PCA-based models in the benchmark — usually 2–4.
4. ****Tab 4 · Models****
 - * Tick the model groups you want. Untick groups you don't need to keep the run fast.
5. ****Tab 5 · Run****
 - * Pick an output directory.
 - * Optionally point to an external predictions folder (.npz files).
 - * Click ****Start benchmark****. The log shows per-model progress.
6. ****Tab 6 · Results****
 - * Leaderboard sorted by ``avg(Val RMSE, Test RMSE)``.
 - * Browse Figures 1–24; click ****Export All Plots as SVG**** to get vector versions; ****Export metrics_global.csv**** for the summary table.

MODELS — HOW EACH IS COMPUTED

=====

Group A (splines / LIR):

- * ****LIR Quadratic / Boltzmann**** — fit $\log(I1/I2)$ vs $1/T$ with a quadratic or two-level Boltzmann form (linear in $1/T$ at high T).
- * ****PC1 polynomial (BIC)**** — polynomial regression of T on PC1; degree is selected by Bayesian Information Criterion.
- * ****PCHIP on PC1**** — monotone piecewise-cubic Hermite interpolation.
- * ****Sensor fusion**** — combines LIR with PC1 via a learned ratio.
- * ****MLR**** — multiple linear regression on all selected PCs.
- * ****Poly(3 PCs, deg 7)**** — multivariate polynomial up to degree 7.
- * ****TPS(3 PCs)**** — thin-plate spline interpolation in PC space.

Group B (trees):

- * Random Forest, Gradient Boosting, XGBoost, LightGBM, ExtraTrees, CatBoost. All operate on the PCA features. Hyper-parameters are chosen on the validation set inside each model.

SpectraSensML v1.0 — User Guide

Group C (kernel / probabilistic):

- * SVR (RBF and polynomial), Gaussian Process Regression (RBF kernel with multiple restarts), k-NN.

Group D (regularised linear):

- * Kernel Ridge (polynomial), PLS (component grid), Bayesian Ridge, ElasticNet ($\alpha \times$ L1-ratio grid), small MLP (sklearn, lbfgs solver).

Group E (deep nets):

- * **ANN-PCA** — MLP on PCA features (multiple architectures).
- * **ANN-SNV** — MLP directly on the SNV-normalised spectrum.
- * **CNN-1D** — small 1-D convolutional network on the SNV spectrum (PyTorch). Defaults are tuned for fast CPU runs.

Optional **virtual models**: any other regressor (e.g. trained externally on GPU) can be added by dropping `.npz` files with predictions into the **External predictions** folder; they appear in the leaderboard alongside the in-process models.

PCA — HOW TO CHOOSE N_COMPONENTS

PCA replaces each spectrum (hundreds or thousands of channels) with a few **Principal Components** (PCs) — uncorrelated linear combinations that explain most of the variance.

- * On the PCA tab you set **n_components to fit** (e.g. 9). The screen reports the cumulative explained variance.
- * **n_components (k) for benchmark** is a **subset** — how many of those PCs to feed the downstream models. Pick the smallest k that captures the temperature-relevant variance.

A useful heuristic:

- * Plot the cumulative explained variance on Figure 4a.
- * Pick k where cumulative variance $\geq 99\%$, **and** where you can still see temperature ordering in the PC scatter (Figure 4c/d).
- * For most luminescence datasets, **k = 2 or 3** is enough. Use more PCs only when models clearly underfit.

METRICS — WHAT EACH NUMBER MEANS

Errors are defined as $e_i = \hat{y}_i - y_i$ (signed; positive = over-predict). All metrics are reported in the variable's unit (default K).

Sample-level (over all spectra in a split):

- * **RMSE** — $\sqrt{\text{mean}(e^2)}$. Penalises large errors. Primary headline metric.
- * **MAE** — $\text{mean}(|e|)$. Robust to outliers.
- * **Bias** — $\text{mean}(e)$. Systematic offset; ideally 0.
- * **Precision** — $\text{std}(e, \text{ddof}=1)$. Spread of the error.
- * **MaxAbs** — $\max(|e|)$. Worst-case error in the split.
- * **P95Abs** — 95th percentile of $|e|$. Robust worst-case bound.

Per set-point (one row per unique T_k):

- * Same six metrics computed inside each T_k bin ($n_k \geq 3$ required for σ).

SpectraSensML v1.0 — User Guide

Across set-points (over the per-T table):

- * **MeanBin_X** – average of X over the unique T_k values.
- * **WorstBin_X** – max of X over T_k (for Bias, max of |Bias|).
This is the metric that tells you "how bad can it get at any single temperature?".

R^2 is intentionally **not** reported – it is a poor figure of merit for sensor accuracy work.

RNG SEED – WHY IT MATTERS

Every random decision in the pipeline (train/val draw, network weight initialisation, RF/XGBoost row subsampling, ...) is seeded by a single integer. Re-running with the same seed and the same data exactly reproduces previous results. Change it and you'll typically see ± 0.1 K RMSE jitter on neural models – useful for assessing how stable a benchmark is.

OUTPUTS – WHAT GETS WRITTEN

A successful run leaves the following inside your chosen output folder:

- * **metrics_global.csv** – one row per (model, set) with all sample-level and Bin-summary metrics.
- * **metrics_per_temperature.csv** – one row per (model, set, T_k) with the six per-bin metrics.
- * **Figure 1.png ... Figure 24.png** – the 24 paper figures.
- * **Figure 1.csv, Figure 3a.csv, ... Figure 24.csv** – the underlying data for every sub-panel, one CSV per panel.
- * **run_info.json** – timestamps, app version, library versions, winner.

Use ***Export All Plots as SVG*** on the Results tab to re-render the same figures as vector graphics for journal-quality editing.

EXTERNAL PREDICTIONS (ADVANCED)

You can mix predictions from models trained **outside** this app – for example a heavy CNN trained on a GPU cluster – by dropping `.npz` files in the ***External predictions*** folder. Each file must contain:

train, val, test_unseen	– 1-D arrays aligned with the dataset rows
model_name	– string
group	– single letter A–E

The benchmark will compute metrics for these "virtual" models and rank them alongside the in-process models. Leave the folder blank if you have none – this is purely opt-in.

CITATION & LICENSE

This software is released under the **GNU General Public License v3.0 (GPLv3)**. If you use it for any published or otherwise disseminated

SpectraSensML v1.0 — User Guide

work – papers, theses, reports, application notes, presentations – you are required to cite the accompanying publication that introduces this tool. See the **About** tab for the canonical citation block.

PER-FIELD REFERENCE

=====

Background subtract

If on, subtract the per-spectrum minimum intensity inside the cropped window before normalisation. Useful when there is a wavelength-independent dark offset.

Figure 3 (LIR diagnostic) always uses background subtraction internally regardless of this checkbox, because LIR is a ratio of band intensities that requires baseline removal.

Export DPI

Resolution used when re-rendering Figure PNGs from the Results tab. 600 DPI is the journal-quality default; drop to 150–300 for quick previews.

External NN/CNN predictions (.npz)

Optional. Folder of .npz files containing precomputed predictions for additional 'virtual' models (e.g. a network trained in another environment). Each file must contain arrays 'train', 'val', 'test_unseen' aligned with the loaded dataset, plus strings 'model_name' and 'group'. Leave blank if you have none – this is purely opt-in.

File extension

Glob pattern selecting which files inside each temperature folder are loaded. Defaults to '*.txt'. Use '*.csv' or '*.asc' if your spectrometer writes a different extension.

Header end marker

Exact line that separates the metadata header from the numeric wavelength–intensity block. Leave blank to auto-detect (the loader will skip lines until it finds the first parseable pair of numbers).

If you supply a marker that does not appear in the file, loading aborts with an error – clear the box for auto-detect.

Integration-time prefix

The header text in each spectrum file immediately followed by the integration time in seconds. Used to divide raw counts by integration time so spectra of unequal exposure are comparable.

If the prefix is missing in a file, integration time defaults to 1.0 (no scaling) and the load continues. Leave blank if all your spectra were taken with the same integration.

LIR bands

Wavelength ranges (in nm) of the two emission bands used to compute the Luminescence Intensity Ratio $LIR = I1 / I2$.

Set the I1 band over the high-energy peak and I2 band over the low-energy peak (or thermally-coupled levels of interest). The intensities are integrated inside each range

SpectraSensML v1.0 — User Guide

after background subtraction.

Model selection

Tick which models to include in the benchmark. Groups:

- A – Splines, polynomial regressions, LIR fits.
- B – Tree ensembles (Random Forest, Gradient Boosting, XGBoost, LightGBM, ExtraTrees, CatBoost).
- C – Kernel / probabilistic (SVR, Gaussian Process, kNN).
- D – Regularised linear (Kernel Ridge, PLS, Bayesian Ridge, ElasticNet, MLP).
- E – Neural networks (ANN on PCA features, ANN on SNV spectrum, 1-D CNN on raw spectrum).

Untick groups you do not need to keep the run short.

Normalisation

How each cropped spectrum is rescaled before modelling.

- SNV – subtract mean and divide by per-spectrum std. Makes models robust to lamp/laser intensity drift; recommended default for sensor work.
- MAX – divide by max intensity in the window. Familiar but sensitive to noise on the peak channel.
- AREA – divide by the integrated area in the window. Stable, good when peak heights vary.
- None – keep raw (after integration-time scaling).

Output directory

Folder where the benchmark writes:

- metrics_global.csv, metrics_per_temperature.csv
- Figure 1.png ... Figure 24.png (or .svg)
- One CSV per sub-panel (Figure 3a.csv, ...)
- run_info.json – provenance for the run.

PCA n_components to fit

How many principal components to compute on the training spectra. The PCA tab will display explained variance and loadings up to this many PCs.

Capped at $\min(50, \text{channels}, n_{\text{train}} - 1)$ automatically.

n_components (k) for benchmark

Number of leading PCs supplied as features to PCA-based models (polynomial regressions, TPS, splines, MLR, ...). Choose just enough to capture the temperature-relevant variance – usually 2–4 PCs are sufficient. Higher values tend to overfit and slow training.

RNG seed

Integer that seeds every random operation in this app: the train/val split, weight initialisation for neural models, and any stochastic optimiser internals.

Pick any number (default 42). Using the same seed on the same data reproduces the same train/val split and the same model weights, so you can re-run a study exactly.

Train fraction (within Train/Val folders)

Fraction of spectra inside each folder marked Train/Val that are kept for training; the

SpectraSensML v1.0 — User Guide

rest become validation. 80 % is a typical default. The split is random but deterministic given the RNG seed.

Variable name

Symbol used on every axis label, colorbar, leaderboard column and CSV header for the temperature-dependent quantity. Defaults to 'T'. Set it to e.g. 'P' if you are training on pressure data, or 'pH', 'B', etc.

Variable unit

Physical unit shown next to the variable name on axes, colorbars, error bars, and metric labels (e.g. 'RMSE (K)' becomes 'RMSE (your-unit)'). Defaults to 'K'.

λ max

Upper wavelength of the spectral window. Symmetric counterpart to λ min.

λ min

Lower wavelength of the spectral window used for every model (crop applied before normalisation). Set so the window covers the emission peaks of interest and excludes detector noise and stray lines.